17 SYNCHRONIZACE

17.1 SYNCHRONIZACE POMOCÍ MONITORŮ

- Monitor je synchronizační nástroj na vysoké úrovni, který umožňuje bezpečné sdílení libovolných dat a usnadňuje programátorovi práci
- Na rozdíl od semaforů se jedná o jazykový konstrukt – tzn. že je zabudován přímo do programovacích jazyků pro paralelní zpracování
- Jeho implementaci musí podporovat samotný překladač

Fungování monitorů je založeno na následujících principech:

17.1.1 VZÁJEMNÉ VYLOUČENÍ

- Monitor zahrnuje vyhrazenou datovou oblast a sadu procedur
- Externí procesy nemají k těmto chráněným datům přímý přístup a mohou s nimi pracovat výhradně prostřednictvím procedur monitoru
- Jádro OS a překladač zajišťují, že v monitoru se v jednom okamžiku může nacházet nejvýše jeden proces (procedury se vzájemně vylučují)
- Pokud je monitor obsazen, další procesy žádající o vstup jsou řazeny do vstupní fronty

17.1.2 PODMÍNKOVÉ PROMĚNNÉ

- Pro složitější synchronizaci procesů uvnitř monitoru se definují tzv. podmínkové proměnné (datový typ `condition`), se kterými jsou sdruženy fronty čekajících procesů

17.1.3 OPERACE WAIT A SIGNAL

- Na podmínkové proměnné (např. `x`) lze aplikovat dvě operace
- Volání `x.wait()` způsobí, že se právě běžící proces uvnitř monitoru zablokuje a monitor uvolní, aby do něj mohl vstoupit jiný proces
- Volání `x.singal()`:
  - aktivuje právě jeden proces, který na tuto podmínku čeká (pokud žádný nečeká, nestane se nic)
  - odblokovaný proces však musí před svým pokračováním monitor nejprve znovu získat

17.2 SYNCHRONIZACE POMOCÍ ZASÍLÁNÍ ZPRÁV

- Zasílání zpráv je velmi flexibilní metoda meziprocesové komunikace, protože jako jediná rovnou kombinuje synchronizaci s výměnou samotných dat
- Její velkou výhodou je možnost využití nejen na jednom stroji, ale i v distribuovaných systémech
- Základem jsou dvě operace:
  - `send(cíl, zpráva)` – pro odeslání
  - `receive(zdroj, zpráva)` – pro přijetí
- Komunikace může být:
  - asynchronní – odesílatel zprávu pošle a nečeká na výsledek
  - synchronní – odesílatel čeká na potvrzení od příjemce

17.2.1 KONCEPTY VYUŽÍVANÉ PŘI ZASÍLÁNÍ ZPRÁV:

17.2.1.1 MAILBOX

- Slouží pro nepřímou komunikaci
- Je to vyrovnávací paměť spravovaná operačním systémem, do které procesy zprávy vkládají a ze které je čtou
- Schránka může být soukromá pro dva procesy nebo sdílená vícero procesy

17.2.1.2 PORT

- Je to specifický typ schránky, který je vlastněný jediným příjemcem (typicky serverem)
- Zprávy do tohoto portu pak může zasílat mnoho různých klientských procesů

17.2.1.3 RENDEZVOUS

- Vzniká tehdy, když má komunikační kanál (schránka) nulovou kapacitu
- Pokud chce odesílatel poslat zprávu a příjemce ještě nevolal funkci pro její výběr, odesílatel se zablokuje a čeká
- Stejně tak čeká příjemce, pokud žádná zpráva nedorazila
- Jakmile se oba sejdou, data se předají a oba procesy následně pokračují nezávisle dál

17.2.2 VZÁJEMNÉ VYLOUČENÍ KRITICKÉ SEKCE ŘEŠENÉ ZASÍLÁNÍM ZPRÁV

Mailbox lze velmi elegantně využít jako zamykací mechanismus (mutex) pro přístup ke sdílenému zdroji:

17.2.2.1 INICIALIZACE

- Systém na začátku odešle do schránky přesně jednu zprávu (např. s textem `go`)

17.2.2.2 VSTUP DO SEKCE

- Před vstupem do kritické sekce zavolá proces synchronní operaci `receive()`
- Ten proces, který zprávu `go` dostane jako první, může vstoupit do kritické sekce
- Ostatní procesy, které zavolaly `receive()`, se zablokují, protože schránka je nyní prázdná

17.2.2.3 VÝSTUP ZE SEKCE

- Při opouštění kritické sekce zavolá proces `send()`, čímž zprávu `go` do schránky vrátí
- Tím se odblokuje jeden z čekajících procesů a cyklus se opakuje